Sistema de Recomendação Híbrido para Plataforma de Ensino

Projeto Prático — AV2 · Desenvolvimento de Sistemas de IA · CEUB

Pedro Rebello Borges de Barros

26 de outubro de 2026

Sumário

Resumo 1
1 Introdução 2
2 Base de dados e análise de qualidade 2
2.1 Os três arquivos 2
2.2 Integridade referencial 2
2.3 Catálogo e perfis replicados 3
2.4 Ausência de sinal nas interações 4
3 Arquitetura do sistema 6
4 Modelos de recomendação 8
4.1 Filtragem baseada em conteúdo 8
4.2 Filtragem colaborativa 8
4.3 Modelo baseado em conhecimento 9
4.4 Hibridização 10
4.5 Modelos de referência 11
5 Metodologia de avaliação 11
6 Resultados nos dados fornecidos 12
7 Validação com sinal controlado 13
8 Métricas online 16
9 Análise de pontos fortes e fracos 17
10 Sugestões de melhorias 18
11 Demonstração funcional 18
12 Conclusão 20
Referências 20
Apêndice A — Precisão, recall e F1 para K = 5, 10 e 20 20
Apêndice B — Reprodutibilidade 21

Resumo

Este trabalho implementa um sistema que recomenda materiais didáticos combinando três fontes: **filtragem por conteúdo** (metadados dos materiais × perfil do aluno), **filtragem colaborativa** (fatoração de matrizes treinada por ALS sobre o histórico de avaliações) e um **modelo baseado em conhecimento** (regras de pré-requisitos, nível e currículo). Um híbrido de quatro etapas pondera as três pontuações, troca os pesos conforme o que se sabe do aluno (*cold start*), aplica restrições de pré-requisito e diversifica a lista com MMR.

A análise dos dados fornecidos mostrou que as interações **não contêm sinal aprendível**: só 2,0% delas ligam um aluno e um material cadastrados, todas as colunas são uniformes e nem o aluno, nem o material, nem a combinação perfil × obra influenciam a nota (13 testes estatísticos, Seção 2). Nesses dados, todos os modelos empatam com o acaso — que é o resultado correto. Para validar a implementação, o sistema foi avaliado também num **cenário com sinal controlado**: os mesmos alunos e materiais, com interações simuladas a partir de uma

utilidade conhecida. Nele, o híbrido alcança NDCG@10 de 0,0033 contra 0,0004 do aleatório, precisão@3 por obra de 0,413 e o menor erro de nota (RMSE 0,928), superando cada componente isolado com significância estatística (*bootstrap* pareado). No teste A/B simulado, o híbrido teve CTR de 28,8%, contra 11,5% do aleatório. O código, os resultados e uma demonstração em Streamlit acompanham este relatório.

1 Introdução

O objetivo é sugerir a cada aluno livros, artigos e vídeos adequados ao seu perfil acadêmico e ao seu histórico. Nenhuma técnica isolada resolve bem esse problema: a filtragem colaborativa precisa de histórico; a de conteúdo depende da qualidade dos metadados e tende a recomendar sempre o mesmo assunto; e regras explícitas não capturam gosto individual. O sistema combina as três, conforme pede o enunciado:

- **Filtragem baseada em conteúdo** — similaridade entre o perfil do aluno (interesses, disciplinas e materiais já avaliados) e os metadados de cada material.
- **Filtragem colaborativa** — padrões de comportamento de alunos com avaliações parecidas, aprendidos por fatoração de matrizes.
- **Modelo baseado em conhecimento** — regras sobre pré-requisitos, nível de dificuldade por período e relevância curricular de cada área para cada curso.

Os entregáveis estão na pasta do projeto: o código-fonte (`recomendador/`, `pipeline.py`, `app.py`, `tests/`), este relatório (`relatorio/`) e a demonstração funcional (`streamlit run app.py`). O `README.md` explica como reproduzir cada resultado.

2 Base de dados e análise de qualidade

2.1 Os três arquivos

Tabela 1: Arquivos fornecidos

Arquivo	Linhas	Conteúdo
`dados_alunos.csv`	10.000	curso, período, disciplinas cursadas e áreas de interesse
`materiais_didaticos.csv`	10.000	título, tipo, área, nível, descrição e autor
`interacoes.csv`	10.000.000	aluno, material, tipo (leitura ou visualização), nota 1–5, duração e data (2023)

Antes de modelar, cada arquivo foi verificado quanto a integridade, duplicação e presença de sinal. Os achados abaixo determinaram várias decisões de projeto.

2.2 Integridade referencial

As interações citam ids de aluno de 1 a 100.000 e de material de 1 a 50.000, mas os cadastros só têm os ids 1 a 10.000. Apenas 200.622 interações (2,0%) ligam um aluno e um material cadastrados — praticamente o que se espera se as duas colunas fossem sorteadas de forma independente (Tabela 2).

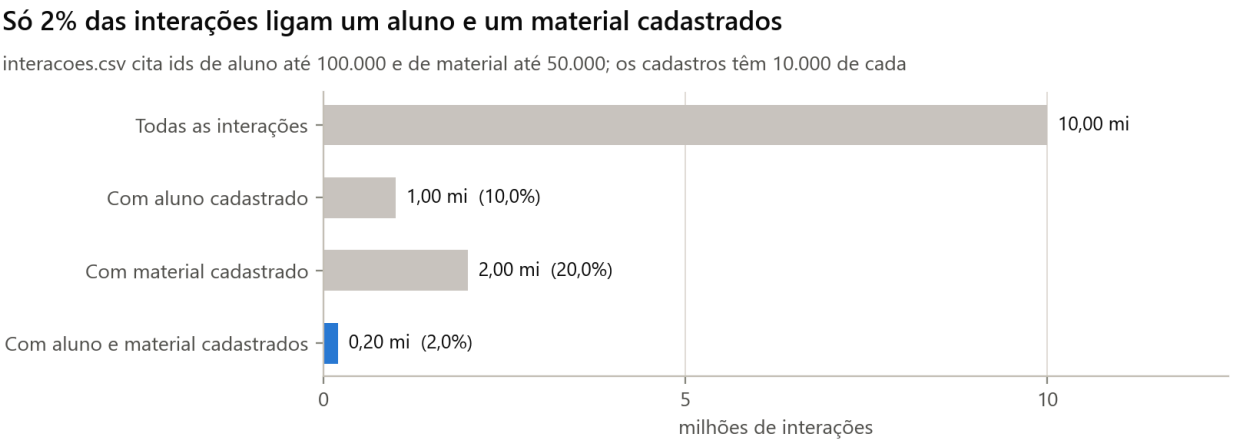


Figura 1: Integridade referencial das interações

Tabela 2: Integridade referencial de interacoes.csv

Recorte	Interações	Parcela
Todas as interações	10.000.000	100%
Com aluno cadastrado	1.000.280	10,0%
Com material cadastrado	1.999.550	20,0%
Com aluno **e** material cadastrados	200.622	2,01%
Esperado para os dois, se independentes	200.011	2,00%

Decisão: nenhuma interação foi descartada. A filtragem colaborativa só precisa de ids, então usa todas as 10 milhões; os modelos de conteúdo e de conhecimento usam apenas o que tem cadastro; e só materiais cadastrados são recomendados, porque um material sem título não pode ser exibido. Alunos sem cadastro continuam atendidos pelo histórico (ver as regras de troca na Seção 4.4).

2.3 Catálogo e perfis replicados

Os 10.000 materiais são **10 obras distintas**, cada uma repetida 1.000 vezes com ids diferentes (Tabela 3), e os 10.000 alunos são **8 perfis** repetidos 1.250 vezes (Tabela 4). Para o modelo de conteúdo, as 1.000 cópias de uma obra são indistinguíveis; só o histórico pode diferenciá-las. Por isso as métricas são reportadas em dois níveis: por material (o item recomendado) e por obra (o que o aluno percebe).

Tabela 3: As 10 obras do catálogo (1.000 cópias de cada) e os pré-requisitos da base de conhecimento

Obra	Tipo	Área	Nível	Autor	Pré-requisitos
Introdução a Algoritmos	livro	Programação	Iniciante	Thomas Cormen	—
Aprendendo Python	livro	Programação	Iniciante	Eric Matthes	—
ML Avançado	livro	IA	Avançado	Aurélien Géron	(Inteligência Artificial ou Machine Learning) e Estatística e Programação
Estatística Básica	livro	Estatística	Iniciante	Mario Triola	—
Deep Learning	vídeo	IA	Intermediário	Andrew Ng	Programação e (Estatística ou Inteligência Artificial ou Machine Learning)
Gestão Ágil	artigo	Gestão	Intermediário	Jeff Sutherland	—
Segurança em Redes	livro	Segurança	Avançado	William Stallings	Redes
UX Moderno	artigo	UX	Intermediário	Don Norman	—
Banco de Dados NoSQL	livro	BD	Intermediário	Martin Fowler	Banco de Dados
IoT Prático	vídeo	IoT	Iniciante	Adafruit	—

Tabela 4: Os 8 perfis acadêmicos distintos de dados_alunos.csv

Curso	Período	Disciplinas cursadas	Interesses	Alunos
Engenharia de Software	5º	Algoritmos, Banco de Dados, IA	ML, Programação, Cloud	1.250
Ciência de Dados	3º	Estatística, Python, ML	IA, Visualização, Big Data	1.250
Engenharia de Computação	7º	Redes, SO, Arquitetura	IoT, Segurança, Hardware	1.250
Sistemas de Informação	4º	Gestão, Redes, BD	UX, PM, Analytics	1.250
Engenharia de Software	6º	BD, IA, Web	Mobile, DevOps, QA	1.250
Ciência de Dados	2º	Python, Estatística	NLP, Deep Learning	1.250
Engenharia de Computação	8º	Arquitetura, Compiladores	Robótica, Embedded	1.250
Sistemas de Informação	5º	UX, PM, BD	Marketing Digital, Analytics	1.250

As abreviações dos perfis foram normalizadas antes do uso — "BD" e "Banco de Dados" aparecem como disciplinas distintas no arquivo, assim como "IA", "ML", "SO", "PM", "QA" e "SI".

2.4 Ausência de sinal nas interações

Todas as colunas de `interacoes.csv` seguem distribuições uniformes (Figura 2): 20% de cada nota, 50% de cada tipo, durações uniformes entre 5 e 240 minutos e datas uniformes ao longo de 2023. Isso por si só não impede a recomendação; o que impede é a independência entre a nota e tudo o mais (Tabela 5):

- **Não há efeito de aluno nem de material.** Na ANOVA com 100 mil grupos, F = 0,996 (p = 0,83); com 50 mil materiais, F = 1,010. A fração da variância explicada (η^2 = 0,0100 e 0,0050) é exatamente a esperada por acaso para esse número de grupos.
- **O passado não prevê o futuro.** A correlação entre a nota média de um aluno no 1º e no 2º semestre é r = −0,0027; para materiais, r = 0,0035 (Figura 3). Um recomendador aprende justamente essa estabilidade.

- **O perfil não interage com o conteúdo.** A nota média de cada combinação perfil × obra fica entre 2,94 e 3,06 (Figura 4), com F = 0,93 e p = 0,65 na ANOVA, e a escolha da obra também independe do perfil.
- **Até as repetições são aleatórias.** Há 9.928 pares aluno-material repetidos, contra 10.000 esperados num sorteio uniforme.

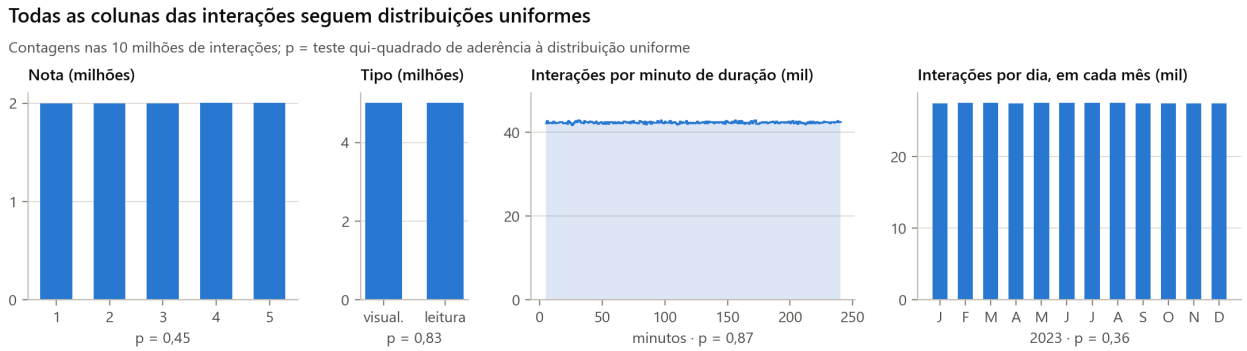


Figura 2: Distribuições das colunas das interações

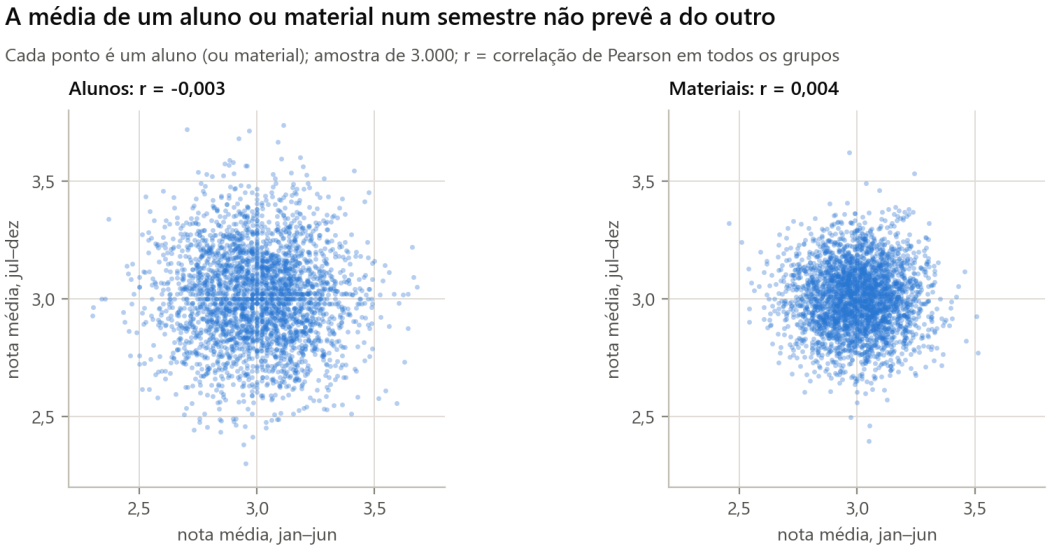


Figura 3: Estabilidade da nota média entre semestres

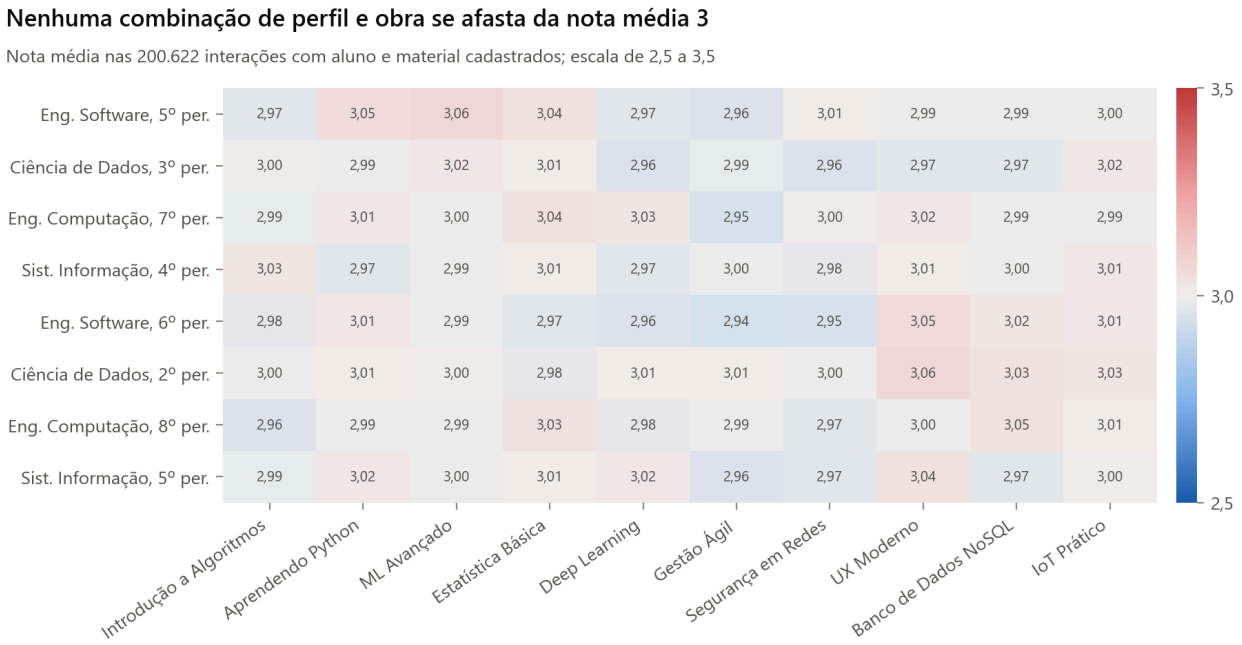


Figura 4: Nota média por perfil e obra

Tabela 5: Testes estatísticos sobre interacoes.csv (10 milhões de linhas)

Hipótese testada	Teste	Estatística	p	Conclusão
Notas uniformes em 1–5	χ^2 aderência	$\chi^2 = 3{,}7$	0,448	uniforme
Tipo de interação 50/50	χ^2 aderência	$\chi^2 = 0{,}04$	0,833	uniforme
Duração uniforme em 5–240 min	χ^2 aderência	$\chi^2 = 211$	0,868	uniforme
Datas uniformes em 2023	χ^2 aderência	$\chi^2 = 373$	0,357	uniforme
Nota × tipo de interação	χ^2 independência	V = 0,0003	0,924	independentes
Nota × mês	χ^2 independência	V = 0,0011	0,327	independentes
Nota × duração	Spearman (2 mi)	$\rho = -0{,}0013$	0,061	sem correlação
Efeito do aluno na nota	ANOVA, 100 mil grupos	F = 0,996	0,834	nenhum
Efeito do material na nota	ANOVA, 50 mil grupos	F = 1,010	0,063	nenhum
Média do aluno, 1º × 2º semestre	Pearson	$r = -0{,}0027$	0,397	sem estabilidade
Média do material, 1º × 2º semestre	Pearson	r = 0,0035	0,429	sem estabilidade
Nota × (perfil, obra)	ANOVA, 80 células	F = 0,933	0,647	nenhum
Escolha da obra × perfil	χ^2 independência	V = 0,0067	0,482	independentes

Com 10 milhões de linhas, esses testes detectariam efeitos minúsculos; não os detectaram. **Conclusão:** as interações são estatisticamente indistinguíveis de ruído uniforme, e nenhum modelo — por melhor que seja — pode superar o acaso nelas. O relatório trata isso como resultado, não como falha: a Seção 6 mostra que todos os modelos empatam com o acaso, e a Seção 7 valida a implementação num cenário em que existe sinal conhecido.

3 Arquitetura do sistema

O sistema é um pacote Python (`recomendador/`) com módulos independentes por responsabilidade, orquestrado por `pipeline.py` e exposto pela demonstração `app.py` (Figura 5).

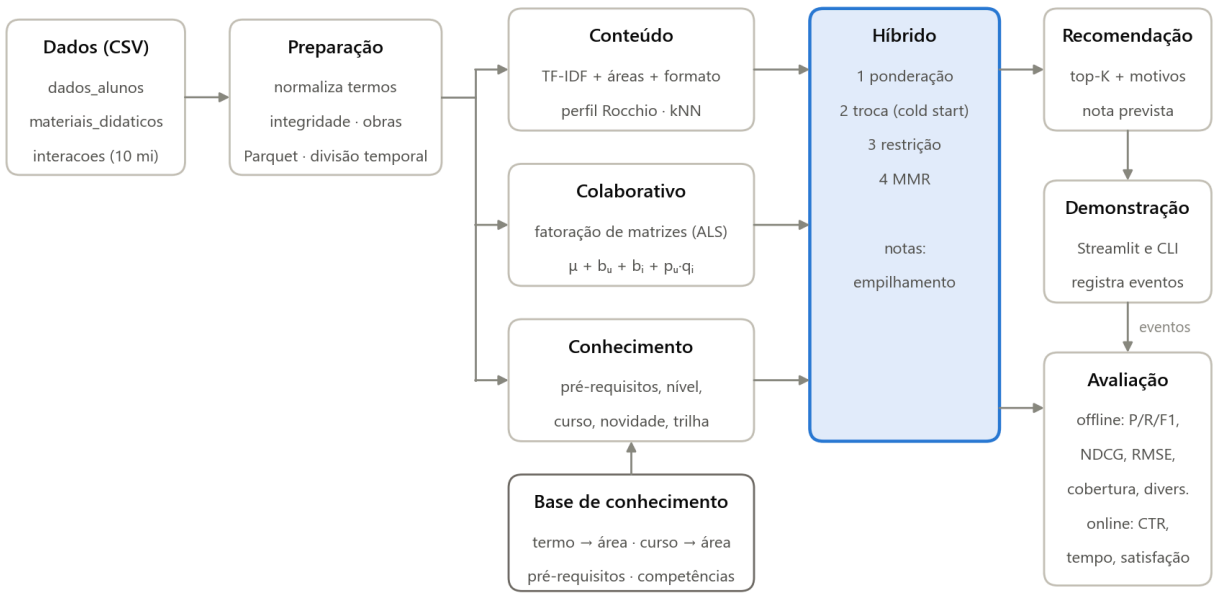


Figura 5: Arquitetura do sistema

Tabela 6: Módulos do pacote `recomendador`

Módulo	Responsabilidade
`dados.py`	Leitura em blocos de 2 milhões de linhas com tipos compactos (160 MB em memória), cache Parquet, normalização de termos, identificação de obras e perfis, divisão temporal
`analise.py`	Integridade referencial e os 13 testes estatísticos da Seção 2
`ontologia.py`	Base de conhecimento escrita à mão: termo ⟶ área, curso ⟶ área, pré-requisitos e competências de cada obra
`modelos/`	Um recomendador por arquivo, todos com a mesma interface: `treinar`, `pontuar`, `prever_avaliacao`, `recomendar`
`avaliacao.py`, `experimentos.py`	Protocolo offline, métricas, intervalos de confiança, ajuste na validação
`simulacao.py`	Cenário com sinal controlado e teste A/B simulado
`online.py`	Registro de eventos da demonstração e cálculo de CTR, engajamento e satisfação
`figuras.py`, `relatorio.py`	Figuras e tabelas deste relatório, geradas a partir dos resultados

Uma recomendação percorre estes passos:

1. Carrega o perfil (se houver) e o histórico do aluno.
2. Cada componente pontua os 10.000 materiais candidatos.
3. As pontuações são normalizadas para [0, 1] por aluno e somadas com os **pesos efetivos** daquele aluno (regras de troca).
4. Materiais cujos pré-requisitos o aluno não cumpre perdem uma penalidade fixa.
5. O MMR escolhe os K itens finais equilibrando relevância e variedade; materiais já vistos são excluídos.
6. Cada item sai com suas justificativas (regras disparadas, termos e área em comum, nota prevista).
7. Na demonstração, impressões, cliques, tempo de estudo e satisfação são registrados para as métricas online.

Decisões de projeto. Todas as operações pesadas são vetorizadas com matrizes esparsas: a filtragem colaborativa treina sobre 8,3 milhões de avaliações em segundos por iteração, e as pontuações são calculadas em lotes de 500 alunos × 10.000 materiais. A base de conhecimento é código, não dados aprendidos — isso torna as regras

auditáveis e permite recomendar a quem não tem histórico. Todo o pipeline usa sementes fixas e é reproduzível com `python pipeline.py`.

4 Modelos de recomendação

4.1 Filtragem baseada em conteúdo

Cada material vira um vetor com três blocos, normalizados separadamente: **área** (one-hot nas 8 áreas), **formato** (tipo e nível, one-hot) e **texto** (TF-IDF de título e descrição, sem acentos, sem *stopwords* e com remoção do plural). A similaridade entre um perfil p e um material x é a média ponderada do cosseno de cada bloco:

$$s_{\text{cont}}(u,i) = \sum_{b} w_b \cos\!\left(p_u^b, x_i^b\right), \qquad b \in \{\text{área, formato, texto}\}$$

com pesos 0,5 para área, 0,2 para formato e 0,3 para texto.

O perfil do aluno soma duas partes no mesmo espaço:

- **declarada:** interesses (peso 1) e disciplinas (peso 0,6) projetados nas áreas pela ontologia — "Deep Learning" → IA, "Analytics" → Estatística — mais o TF-IDF desses termos;
- **histórica (Rocchio):** a média dos vetores dos materiais avaliados, com peso (nota − 3)/2, positiva para notas 4–5 e negativa para 1–2.

$$p_u = \hat{d}_u + \frac{n_u}{n_u + 5}\,\hat{h}_u, \qquad h_u = \sum_j \frac{r_{uj} - 3}{2}\,x_j$$

em que d_u é o perfil declarado, o acento circunflexo indica normalização (norma 1) e n_u é o tamanho do histórico. Assim, um aluno novo é descrito só pelo que declarou, e o histórico ganha peso à medida que cresce. Para prever notas, o componente usa um **kNN de conteúdo**: a nota prevista é o baseline $b_{ui} = \mu + b_u + b_i$ corrigido pelos resíduos que o aluno deu a materiais parecidos, com similaridade elevada ao quadrado e encolhimento:

$$\hat{r}_{ui} = b_{ui} + \frac{\sum_j s_{ij}^2 \left(r_{uj} - b_{uj}\right)}{\sum_j s_{ij}^2 + 1}$$

4.2 Filtragem colaborativa

Fatoração de matrizes com vieses (Koren, Bell e Volinsky, 2009):

$$\hat{r}_{ui} = \mu + b_u + b_i + p_u^{\top} q_i, \qquad \min \sum_{(u,i)} \left(r_{ui} - \hat{r}_{ui}\right)^2 + \lambda \left(\|p_u\|^2 + \|q_i\|^2 \right) + \lambda_b \left(b_u^2 + b_i^2 \right)$$

com 8 fatores, $\lambda = 5$ e $\lambda_b = 10$. O treino é por **mínimos quadrados alternados** (ALS): com os materiais fixos, o vetor $[p_u, b_u]$ de cada aluno é a solução exata de uma regressão *ridge*; depois o mesmo para os materiais. A implementação é vetorizada — as matrizes de Gram de todos os alunos saem de um único produto esparso e os sistemas 9×9 são resolvidos em lote —, o que permite treinar sobre as 10 milhões de interações. A validação escolhe o número de iterações (parada antecipada com paciência 8, guardando a melhor iteração, porque a curva de validação do ALS pode piorar antes de melhorar; Figura 6).

Alunos com vetores p_u próximos têm padrões de avaliação parecidos: é assim que o modelo usa "usuários similares", e a demonstração mostra esses vizinhos. Uma primeira versão treinada por SGD ficou presa no nível dos vieses: nenhuma das 36 combinações de hiperparâmetros testadas tirou os fatores de perto de zero antes da parada antecipada, e o modelo recomendava os mesmos 13 materiais a todos. A troca pelo ALS baixou o RMSE de validação do cenário controlado de 1,130 para 1,074, elevou o NDCG@10 de 0,0014 para 0,0021 e a cobertura do catálogo de 0,1% para 11%. Regularização forte ($\lambda \geq 10$) também colapsa os fatores, por isso $\lambda = 5$.

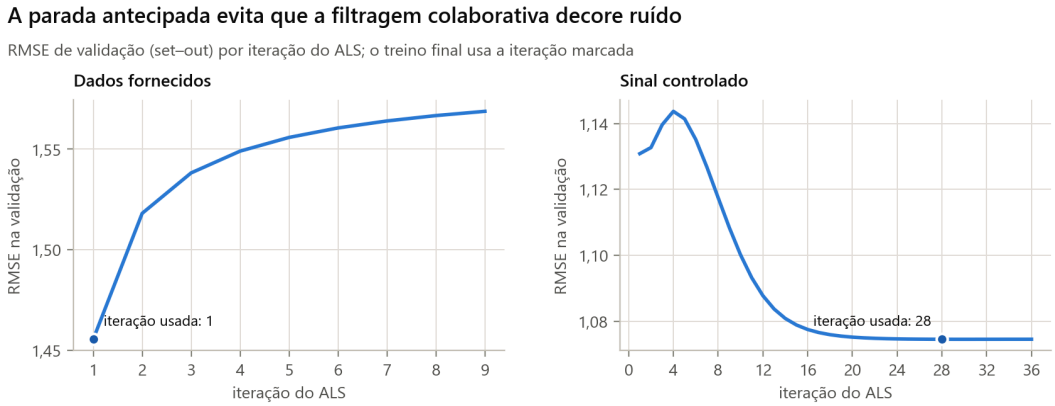


Figura 6: Curva de validação da filtragem colaborativa

4.3 Modelo baseado em conhecimento

A base de conhecimento (`ontologia.py`) define, para cada obra, os pré-requisitos e as competências que ela ensina (Tabela 3), e para cada curso a relevância de cada área. As competências do aluno vêm das disciplinas cursadas (com implicações: cursar Python comprova base de Programação) e das obras concluídas no histórico (nota ≥ 3). Cinco regras são aplicadas a cada par aluno × obra:

1. **Pré-requisitos** — todos os grupos exigidos precisam ser atendidos; caso contrário a obra é inelegível (pontuação 0).
2. **Curso** — relevância curricular da área (ex.: IA vale 1,0 em Ciência de Dados e 0,3 em Sistemas de Informação).
3. **Nível** — 1,0 no nível esperado para o período (1º–2º: iniciante; 3º–5º: intermediário; 6º+: avançado), 0,6 um degrau acima ou abaixo, 0,2 dois degraus.
4. **Novidade** — obra introdutória de algo que o aluno já cursou é redundante.
5. **Desbloqueio** — obra que ensina um pré-requisito que falta para outra obra de uma área de interesse do aluno (trilha de aprendizagem).

$$s_{\text{conh}}(u, w) = \text{elegível}\ (u, w) \times (\alpha_1\ \text{curso} + \alpha_2\ \text{nível} + \alpha_3\ \text{novidade} + \alpha_4\ \text{desbloqueio})$$

com pesos 0,35 para curso e nível e 0,15 para novidade e desbloqueio.

Toda pontuação vem acompanhada das regras que a produziram. A saída abaixo, gerada pela demonstração de linha de comando sobre os dados fornecidos, mostra as cinco regras para o aluno 1 (Engenharia de Software, 5º período, interesse em Machine Learning). *ML Avançado* fica com pontuação zero porque falta Estatística, e por isso *Estatística Básica* ganha o bônus de trilha ("Desbloqueia: ML Avançado"); *Introdução a Algoritmos* perde o ponto de novidade porque o aluno já cursou Algoritmos; e as competências vindas do histórico (obras concluídas com nota ≥ 3) também contam como pré-requisitos atendidos.

```
$ python recomendar.py --aluno 1 --regras --k 3
Aluno 1
  Engenharia de Software, 5º período
  Disciplinas: Algoritmos, Banco de Dados, Inteligência Artificial
  Interesses:  Machine Learning, Programação, Cloud
  Competências pelo histórico: Deep Learning, Python, UX
  Histórico: 106 interações, 16 com material cadastrado
  Pesos efetivos: conteúdo 40% · colaborativo 30% · conhecimento 30%

Regras de conhecimento por obra (pontuação 0–1; 0 = pré-requisito faltante)

  0,78  Banco de Dados NoSQL
        Pré-requisitos atendidos · Área central do curso Engenharia de Software · Nível
        Intermediário adequado ao 5º período
  0,71  Aprendendo Python
        Área central do curso Engenharia de Software · Nível Iniciante (esperado para o
```

```
       5º período: Intermediário)
  0,71  Gestão Ágil
       Área complementar ao curso Engenharia de Software · Nível Intermediário adequado
       ao 5º período
  0,67  UX Moderno
       Área complementar ao curso Engenharia de Software · Nível Intermediário adequado
       ao 5º período
  0,64  Deep Learning
       Pré-requisitos atendidos · Área complementar ao curso Engenharia de Software ·
       Nível Intermediário adequado ao 5º período
  0,58  Estatística Básica
       Pouca relação com o curso Engenharia de Software · Nível Iniciante (esperado
       para o 5º período: Intermediário) · Desbloqueia: ML Avançado
  0,56  Introdução a Algoritmos
       Área central do curso Engenharia de Software · Nível Iniciante (esperado para o
       5º período: Intermediário) · Conteúdo de disciplina já cursada
  0,40  IoT Prático
       Pouca relação com o curso Engenharia de Software · Nível Iniciante (esperado
       para o 5º período: Intermediário)
  0,00  ML Avançado
       Faltam pré-requisitos: Estatística · Área complementar ao curso Engenharia de
       Software · Nível Avançado (esperado para o 5º período: Intermediário)
  0,00  Segurança em Redes
       Faltam pré-requisitos: Redes · Pouca relação com o curso Engenharia de Software
       · Nível Avançado (esperado para o 5º período: Intermediário)

Top 3 — Híbrido

 1. Introdução a Algoritmos — livro, Programação, Iniciante (material 2721)
      nota prevista 4,0 · Área central do curso Engenharia de Software · Nível Iniciante
      (esperado para o 5º período: Intermediário) · Conteúdo de disciplina já cursada
 2. Banco de Dados NoSQL — livro, BD, Intermediário (material 7129)
      nota prevista 3,8 · Pré-requisitos atendidos · Área central do curso Engenharia de
      Software · Nível Intermediário adequado ao 5º período
 3. Aprendendo Python — livro, Programação, Iniciante (material 9062)
      nota prevista 3,9 · Área central do curso Engenharia de Software · Nível Iniciante
      (esperado para o 5º período: Intermediário)
```

4.4 Hibridização

O híbrido combina as três pontuações em quatro etapas:

1. **Ponderação.** Cada componente é normalizado para [0, 1] por aluno (min-max) e somado com pesos w_c.
2. **Troca (cold start).** Os pesos mudam conforme o que se sabe do aluno: com menos de 5 interações, o peso colaborativo cai na proporção (com zero, some); sem perfil cadastrado, o peso de conhecimento é zero; sem perfil e sem histórico, o sistema recorre à popularidade. Os pesos são renormalizados para somar 1.
3. **Restrição.** Materiais inelegíveis perdem 0,5 ponto — continuam disponíveis se nada melhor existir, mas raramente entram na lista.
4. **Diversificação (MMR).** A lista é montada item a item, maximizando $\lambda \cdot \text{relevância} - (1 - \lambda) \cdot \max_{j \in L} \text{sim}\ (i, j)$, com λ = 0,8 e similaridade de conteúdo. O MMR roda sobre todos os candidatos: com 1.000 cópias por obra, os 100 primeiros costumam ser a mesma obra e um corte prévio impediria a diversificação.

$$s_{\text{híbr}}(u, i) = \sum_c w_c(u)\, \tilde{s}_c(u, i) - \pi \cdot \mathbb{1}[\, i \ \text{ inelegível para } \ u\,]$$

em que $\tilde{s}_c$ é a pontuação normalizada do componente c, $w_c(u)$ é o peso efetivo para o aluno e π = 0,5 é a penalidade.

Os pesos padrão (conteúdo 0,4; colaborativo 0,3; conhecimento 0,3) são substituídos pelo melhor ponto de uma grade de 36 combinações (passo 0,1, nenhum peso abaixo de 0,1) **somente se** a melhora de NDCG@10 na validação for significativa por *bootstrap* pareado entre alunos — caso contrário, a "melhor" combinação seria só ruído de amostragem. Para prever notas, o híbrido usa **empilhamento linear**: a nota é o baseline mais uma combinação dos desvios previstos pelos três componentes, com coeficientes estimados por mínimos quadrados na validação.

4.5 Modelos de referência

Todo resultado é comparado com: **aleatório** (ordem sorteada), **popularidade** (mais avaliações positivas no treino), **média global** e **vieses** $\mu + b_u + b_i$ regularizados (para notas). No cenário controlado há ainda o **oráculo**, que ordena pela utilidade verdadeira do simulador e marca o teto alcançável.

5 Metodologia de avaliação

Divisão temporal. Treino com janeiro–outubro de 2023 e teste com novembro–dezembro, como numa plataforma real (o modelo nunca vê o futuro). Dentro do treino, janeiro–agosto × setembro–outubro serve de validação para o número de iterações da filtragem colaborativa, os pesos do híbrido e os coeficientes do empilhamento; depois os modelos são retreinados em janeiro–outubro.

Protocolo de ranqueamento. São avaliados os alunos cadastrados com ao menos um material relevante no teste (nota ≥ 4). Os candidatos são os 10.000 materiais cadastrados, excluídos os já vistos no treino. As métricas são calculadas por aluno e promediadas:

$$\text{P@K} = \frac{|L_K \cap R|}{K}, \qquad \text{R@K} = \frac{|L_K \cap R|}{|R|}, \qquad \text{F1@K} = \frac{2\,\bar{P}\,\bar{R}}{\bar{P} + \bar{R}}, \qquad \text{NDCG@K} = \frac{1}{\text{IDCG}_K} \sum_{k=1}^{K} \frac{\mathbb{1}[L_k \in R]}{\log_2(k+1)}$$

em que L_K é a lista recomendada, R o conjunto relevante e $\bar{P}$, $\bar{R}$ as médias de P@K e R@K entre os alunos. No **nível de obra**, cada lista é convertida na ordem das obras (pela maior pontuação entre as cópias) e é relevante a obra de qualquer material relevante; usa-se K = 3.

Catálogo. Cobertura = fração dos materiais candidatos que aparecem em alguma lista top-10. Diversidade = média, por lista, de 1 − cos de conteúdo entre os pares de itens (0 quando a lista só tem cópias da mesma obra).

Notas. RMSE e MAE sobre os pares do teste em que aluno e material estão cadastrados (comuns a todos os modelos) e, para os modelos que não dependem de cadastro, sobre todo o teste.

Incerteza. Intervalos de confiança de 95% por *bootstrap* (1.000 reamostragens de alunos).

Dois cenários. Como os dados fornecidos não têm sinal, os mesmos modelos e o mesmo protocolo foram aplicados a um **cenário com sinal controlado** (`simulacao.py`): os mesmos 10.000 alunos e 10.000 materiais, com ~250 mil interações geradas a partir de uma utilidade desconhecida pelos modelos,

$$u(a, m) = \beta_1\,\text{tópico} + \beta_2\,\text{adequação} + \beta_3\,\text{latente} + \beta_4\,\text{qualidade}$$

com pesos 1,0; 0,8; 0,8 e 0,4, cada termo padronizado antes da soma. Nela, *tópico* é o gosto do perfil pela área da obra (70% vindo dos interesses declarados, 30% idiossincrasia sorteada por perfil), *adequação* reúne pré-requisitos, nível e redundância, *latente* é o produto de vetores aleatórios de aluno e material (gosto individual que nenhum metadado explica) e *qualidade* é um viés por material. Cada aluno faz cerca de 25 interações; 70% escolhidas com probabilidade proporcional a e^u e 30% ao acaso; a nota é 3 + 1,1·z(u) + viés do aluno + ruído, arredondada e limitada a 1–5, em que z(u) é a utilidade padronizada. O cenário não prova qual componente venceria com dados reais — o simulador assume, por exemplo, que interesses declarados e pré-requisitos importam. Ele verifica que cada componente captura o tipo de sinal para o qual foi projetado e que o híbrido os combina.

6 Resultados nos dados fornecidos

Foram avaliados 7.394 alunos, com 8.331.775 interações de treino e 1.668.225 de teste. Como previsto pela Seção 2, **nenhum modelo se distingue do aleatório**: todos os intervalos de confiança se sobrepõem ao dele em NDCG@10 e em precisão por obra (Figura 7, Tabela 7).

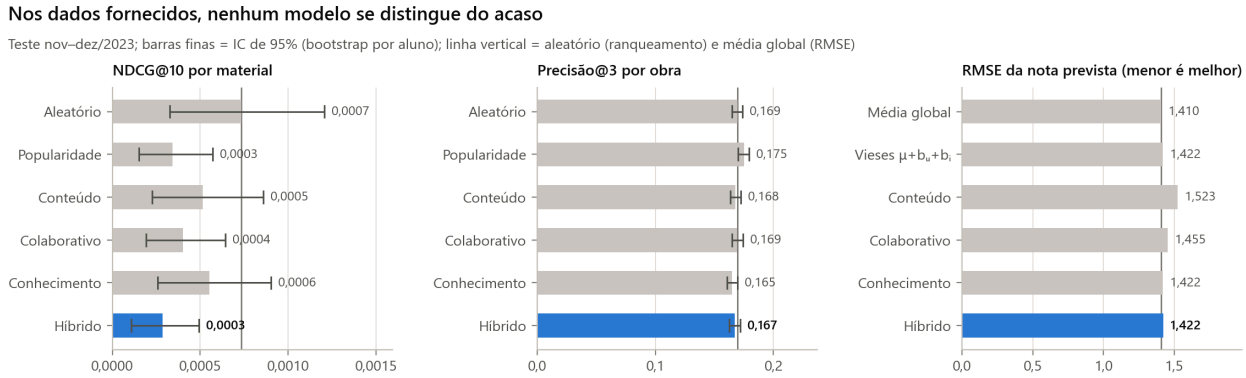


Figura 7: Comparação dos modelos nos dados fornecidos

Tabela 7: Ranqueamento nos dados fornecidos (teste nov–dez/2023; 10 mil materiais candidatos)

Modelo	P@10	R@10	F1@10	NDCG@10	IC 95%	Obra P@3	Cobertura	Diversidade
Aleatório	0,0002	0,0011	0,0004	0,0007	0,0003–0,0012	0,169	99,9%	0,82
Popularidade	0,0001	0,0007	0,0002	0,0003	0,0002–0,0006	0,175	0,1%	0,75
Conteúdo	0,0002	0,0009	0,0003	0,0005	0,0002–0,0009	0,168	99,9%	0,00
Colaborativo	0,0002	0,0009	0,0003	0,0004	0,0002–0,0006	0,169	21,4%	0,81
Conhecimento	0,0002	0,0011	0,0003	0,0006	0,0003–0,0009	0,165	69,6%	0,04
Híbrido	**0,0001**	**0,0006**	**0,0002**	**0,0003**	**0,0001–0,0005**	**0,167**	**40,2%**	**0,58**

Tabela 8: Erro na previsão de notas nos dados fornecidos — 33.471 pares (cadastrados) e 1.668.225 pares (todo o teste)

Modelo	RMSE (cadastrados)	MAE (cadastrados)	RMSE (todo o teste)	MAE (todo o teste)
Média global	1,4101	1,1961	1,4136	1,1993
Vieses $\mu + b_u + b_i$	1,4218	1,2243	1,4244	1,2271
Conteúdo	1,5231	1,2932	—	—
Colaborativo	1,4548	1,2530	1,4565	1,2549
Conhecimento	1,4218	1,2243	—	—
Híbrido	**1,4218**	**1,2244**	—	—

Intervalos que se sobrepõem não bastam para afirmar empate, por isso o híbrido foi comparado com cada modelo aluno a aluno, por *bootstrap* pareado (Tabela 9). Das 10 diferenças, 2 passam do limiar de 5% isoladamente (menor p = 0,015), ambas com o híbrido abaixo do outro modelo (−0,00045 de NDCG@10 frente ao aleatório e −0,0079 de precisão por obra frente à popularidade). Isso é compatível com o acaso em 10 testes — espera-se meio falso positivo. Com a correção de Bonferroni (limiar 0,005), nenhuma diferença é significativa.

Tabela 9: Comparações pareadas do híbrido com cada modelo nos dados fornecidos

Híbrido menos	Métrica	Diferença média	IC 95% pareado	p	Significativa
Aleatório	NDCG@10	−0,00045	−0,00093 a −0,00001	0,042	sim
Popularidade	NDCG@10	−0,00006	−0,00035 a +0,00023	0,707	não
Conteúdo	NDCG@10	−0,00023	−0,00059 a +0,00011	0,208	não
Colaborativo	NDCG@10	−0,00012	−0,00037 a +0,00012	0,343	não
Conhecimento	NDCG@10	−0,00027	−0,00067 a +0,00011	0,187	não
Aleatório	Obra P@3	−0,0023	−0,0080 a +0,0037	0,441	não
Popularidade	Obra P@3	−0,0079	−0,0142 a −0,0016	0,015	sim
Conteúdo	Obra P@3	−0,0005	−0,0038 a +0,0029	0,794	não
Colaborativo	Obra P@3	−0,0020	−0,0077 a +0,0038	0,468	não
Conhecimento	Obra P@3	+0,0021	−0,0024 a +0,0064	0,344	não

Três observações confirmam que o sistema se comportou corretamente diante de ruído:

- **Na previsão de notas, nada supera a média global** (RMSE 1,410, praticamente $\sqrt{2} \approx 1{,}414$, o desvio-padrão de uma nota uniforme em 1–5). Os vieses por aluno e material (RMSE 1,422) e o kNN de conteúdo pioram o erro: cada parâmetro estimado a partir de ruído só adiciona variância.
- **A validação protegeu os modelos.** A filtragem colaborativa parou na iteração 1 (Figura 6), e a busca de pesos não encontrou combinação significativamente melhor que a padrão, que foi mantida (Tabela 10).
- **As diferenças de cobertura e diversidade são estruturais, não de acurácia.** Conteúdo e conhecimento dão a mesma pontuação às 1.000 cópias de uma obra e geram listas de uma obra só (diversidade ≈ 0). A cobertura alta do conteúdo (99,9%) é artefato do desempate aleatório entre cópias idênticas: cada aluno recebe cópias diferentes da mesma obra. O MMR do híbrido eleva a diversidade para 0,58.

Tabela 10: Escolhas feitas na validação (set–out): pesos do híbrido (conteúdo / colaborativo / conhecimento) e iterações da FC

Cenário	Melhor da grade	Ganho de NDCG@10 (IC 95%)	Pesos adotados	Iterações da FC
Dados fornecidos	0,3 / 0,1 / 0,6	+0,00015 (−0,00018 a +0,00049)	0,4 / 0,3 / 0,3	1
Sinal controlado	0,8 / 0,1 / 0,1	+0,00022 (−0,00014 a +0,00061)	0,4 / 0,3 / 0,3	28

7 Validação com sinal controlado

Com sinal presente, os componentes se separam do acaso e o **híbrido supera cada um deles** (Figura 8, Tabela 11). O oráculo mostra o teto: mesmo conhecendo a utilidade verdadeira, a precisão por material é baixa, porque cada aluno tem poucos itens relevantes entre 10.000 candidatos e a escolha simulada tem um componente aleatório.

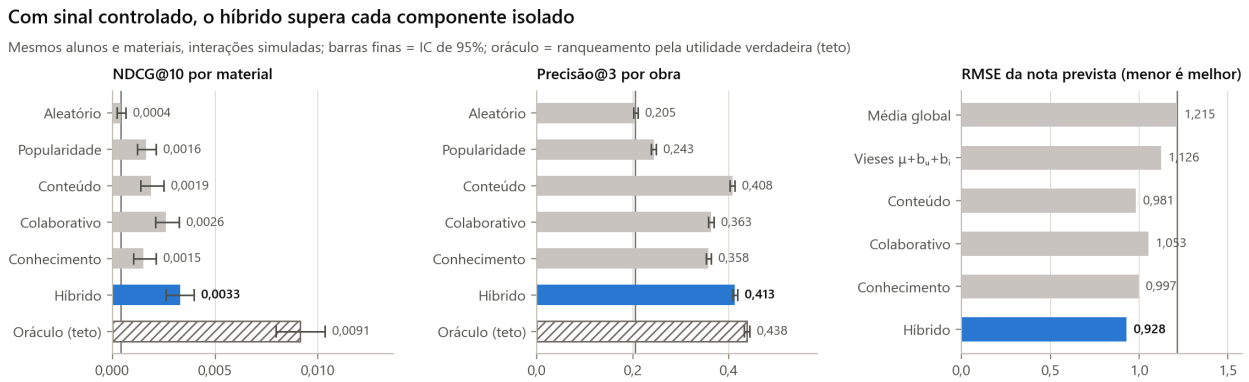


Figura 8: Comparação dos modelos no cenário com sinal controlado

Tabela 11: Ranqueamento no cenário com sinal controlado

Modelo	P@10	R@10	F1@10	NDCG@10	IC 95%	Obra P@3	Cobertura	Diversidade
Aleatório	0,0002	0,0007	0,0003	0,0004	0,0002–0,0007	0,205	100,0%	0,82
Popularidade	0,0007	0,0028	0,0011	0,0016	0,0012–0,0021	0,243	0,1%	0,34
Conteúdo	0,0007	0,0029	0,0012	0,0019	0,0014–0,0025	0,408	99,1%	0,00
Colaborativo	0,0011	0,0043	0,0018	0,0026	0,0021–0,0032	0,363	13,4%	0,46
Conhecimento	0,0005	0,0023	0,0008	0,0015	0,0010–0,0021	0,358	69,4%	0,03
Híbrido	**0,0014**	**0,0054**	**0,0022**	**0,0033**	**0,0026–0,0040**	**0,413**	**21,1%**	**0,42**
Oráculo (teto)	0,0038	0,0150	0,0061	0,0091	0,0080–0,0103	0,438	15,2%	0,30

Tabela 12: Erro na previsão de notas no cenário controlado — 41.601 pares (cadastrados)

Modelo	RMSE	MAE
Média global	1,2152	1,0469
Vieses $\mu + b_u + b_i$	1,1257	0,9327
Conteúdo	0,9811	0,7790
Colaborativo	1,0533	0,8621
Conhecimento	0,9966	0,8008
Híbrido	**0,9278**	**0,7340**

As comparações pareadas confirmam que a vantagem não é ruído: o híbrido supera cada um dos cinco modelos nas duas métricas, nas 10 de 10 comparações, todas com $p \leq 0{,}005$ — inclusive o colaborativo em NDCG@10 (+0,00069; IC +0,00022 a +0,00118) e o conteúdo na precisão por obra (+0,0051; IC +0,0025 a +0,0078).

Tabela 13: Comparações pareadas do híbrido com cada modelo no cenário controlado

Híbrido menos	Métrica	Diferença média	IC 95% pareado	p	Significativa
Aleatório	NDCG@10	+0,00289	+0,00217 a +0,00366	< 0,001	sim
Popularidade	NDCG@10	+0,00167	+0,00081 a +0,00250	< 0,001	sim
Conteúdo	NDCG@10	+0,00142	+0,00050 a +0,00231	0,001	sim
Colaborativo	NDCG@10	+0,00069	+0,00022 a +0,00118	0,005	sim
Conhecimento	NDCG@10	+0,00178	+0,00082 a +0,00271	< 0,001	sim
Aleatório	Obra P@3	+0,2074	+0,2017 a +0,2134	< 0,001	sim
Popularidade	Obra P@3	+0,1695	+0,1632 a +0,1755	< 0,001	sim
Conteúdo	Obra P@3	+0,0051	+0,0025 a +0,0078	< 0,001	sim
Colaborativo	Obra P@3	+0,0497	+0,0446 a +0,0546	< 0,001	sim
Conhecimento	Obra P@3	+0,0548	+0,0512 a +0,0583	< 0,001	sim

Cada componente capturou o sinal para o qual foi projetado:

- **Conteúdo** é o melhor componente isolado no nível de obra (precisão@3 de 0,408): identifica as áreas de que o perfil gosta, mas recomenda dez cópias da mesma obra (diversidade 0,00).
- **Conhecimento** acerta obras adequadas ao momento acadêmico (precisão@3 de 0,358) e funciona sem nenhum histórico.
- **Colaborativo** é o melhor componente isolado no nível de material (NDCG@10 de 0,0026), porque é o único que distingue cópias da mesma obra — pelo gosto latente de cada aluno e pela qualidade de cada material. No

nível de obra fica atrás de conteúdo e conhecimento (precisão@3 de 0,363): não sabe que as 1.000 cópias são a mesma obra e precisa inferir isso de cerca de 20 avaliações por material.

- **Híbrido** junta o melhor de cada um: NDCG@10 de 0,0033 (27% acima do melhor componente, o colaborativo, e 8,1 vezes o aleatório), a maior precisão por obra (0,413) e o menor RMSE (0,928, contra 1,215 da média global), com diversidade de 0,42.

Cold start. Para 2.000 alunos sorteados, todo o histórico de treino foi apagado e os modelos foram retreinados sem eles (Figura 9, Tabela 14). A filtragem colaborativa não tem o que aprender desses alunos; o híbrido, pelas regras de troca, passa a usar só conteúdo e conhecimento e mantém a precisão por obra.

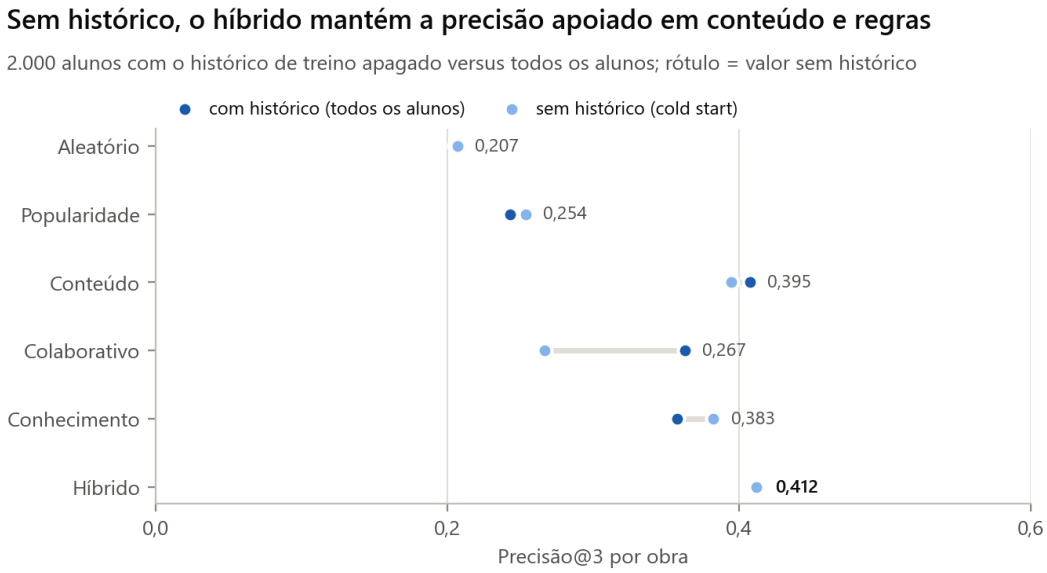


Figura 9: Cold start: precisão por obra com e sem histórico

Tabela 14: Cold start: 2.000 alunos com todo o histórico de treino apagado

Modelo	NDCG@10 sem histórico	Obra P@3 sem histórico	Obra P@3 com histórico	Cobertura sem histórico
Aleatório	0,0015	0,207	0,205	83,1%
Popularidade	0,0015	0,254	0,243	0,1%
Conteúdo	0,0021	0,395	0,408	63,4%
Colaborativo	0,0021	0,267	0,363	0,1%
Conhecimento	0,0021	0,383	0,358	47,0%
Híbrido	**0,0023**	**0,412**	**0,413**	**67,7%**

Diversidade × precisão. O λ do MMR controla a troca entre relevância e variedade (Figura 10, Tabela 15). O valor adotado, 0,8, multiplica a diversidade por 8 (de 0,05 para 0,42) ao custo de 5% do NDCG@10; abaixo dele a perda acelera (com λ = 0,7, 24%). Uma plataforma real calibraria λ com o teste A/B, porque o efeito da diversidade na satisfação não aparece nas métricas offline.

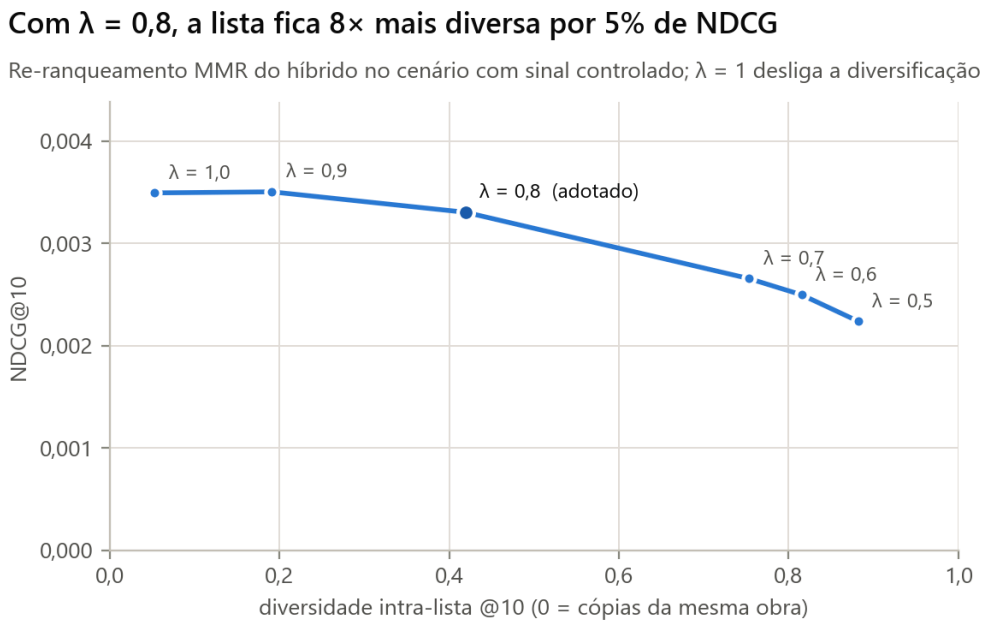


Figura 10: Curva diversidade × NDCG do MMR

Tabela 15: Efeito do λ do MMR no híbrido (cenário controlado)

λ	P@10	NDCG@10	Diversidade@10	Cobertura@10
1,0	0,0014	0,0035	0,052	24,7%
0,9	0,0015	0,0035	0,191	23,2%
0,8	0,0014	0,0033	0,419	21,1%
0,7	0,0011	0,0027	0,753	17,9%
0,6	0,0010	0,0025	0,815	17,1%
0,5	0,0009	0,0022	0,882	16,1%

8 Métricas online

Métricas online exigem usuários reais interagindo com o sistema. O projeto implementa a instrumentação e mede as três métricas pedidas de duas formas.

Na demonstração. Cada lista exibida registra uma *impressão* por item; "Abrir material" registra um *clique*; "Concluí" registra os *minutos de estudo*; e as estrelas registram a *pesquisa de satisfação* (1–5). Os eventos vão para `artefatos/eventos_online.csv` e a aba "Métricas online" calcula, por modelo:

$$\text{CTR} = \frac{\text{cliques}}{\text{impressões}}, \qquad \text{engajamento} = \frac{\text{minutos registrados}}{\text{sessões}}, \qquad \text{satisfação} = \text{média das respostas}$$

No modo "Teste A/B", cada sessão recebe um modelo sorteado, que é o desenho de um experimento controlado real.

Teste A/B simulado. Para comparar os modelos sem usuários reais, alunos simulados do cenário controlado receberam 5.000 sessões por modelo, com 10 recomendações cada. O aluno simulado examina a posição k com probabilidade $1/\log_2(k+1)$, clica com probabilidade crescente na utilidade verdadeira do item, gera minutos de estudo proporcionais a ela e responde à pesquisa (30% das sessões) com uma nota que sobe com a utilidade média dos itens examinados e com a variedade de obras da lista (Figura 11, Tabela 16).

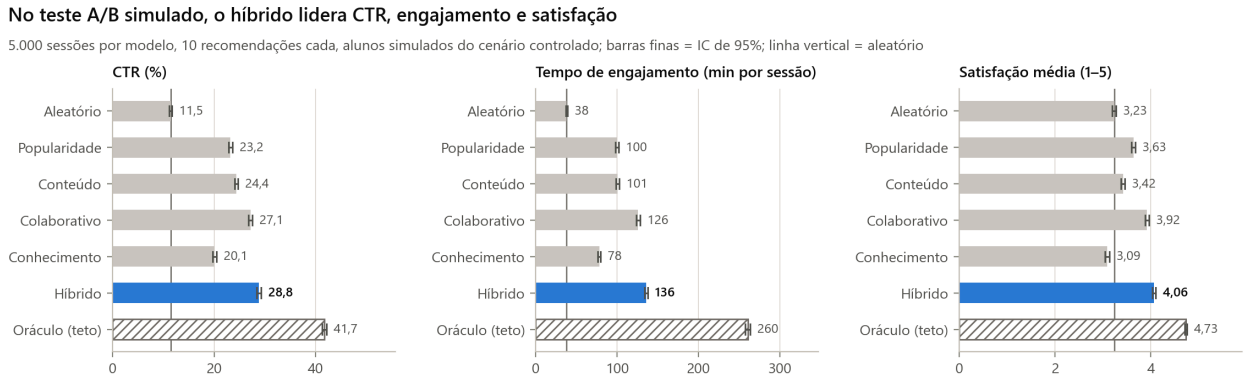


Figura 11: Teste A/B simulado

Tabela 16: Teste A/B simulado: métricas online por modelo (± = meia-largura do IC 95%)

Modelo	CTR	Minutos por sessão	Satisfação (1–5)	Respostas
Aleatório	11,5% ± 0,3	38 ± 1	3,23 ± 0,04	1.487
Popularidade	23,2% ± 0,4	100 ± 2	3,63 ± 0,04	1.513
Conteúdo	24,4% ± 0,4	101 ± 2	3,42 ± 0,04	1.521
Colaborativo	27,1% ± 0,4	126 ± 2	3,92 ± 0,04	1.554
Conhecimento	20,1% ± 0,4	78 ± 2	3,09 ± 0,05	1.474
Híbrido	**28,8% ± 0,4**	**136 ± 2**	**4,06 ± 0,04**	**1.479**
Oráculo (teto)	41,7% ± 0,4	260 ± 3	4,73 ± 0,03	1.469

O híbrido teve CTR de 28,8%, 136 minutos por sessão e satisfação 4,06 — à frente do segundo colocado, o colaborativo (27,1%, 126 minutos e 3,92), sem sobreposição dos intervalos de confiança. Conteúdo e conhecimento têm CTR razoável, mas as menores satisfações entre os modelos personalizados: suas listas repetem uma única obra, e o aluno simulado valoriza variedade. Esses números dependem das hipóteses do aluno simulado; o protocolo de um teste real está nas sugestões de melhoria.

9 Análise de pontos fortes e fracos

Pontos fortes

- **Robusto ao cold start.** Conteúdo e regras funcionam sem histórico, e as regras de troca redistribuem os pesos automaticamente; alunos sem cadastro continuam atendidos pelo histórico.
- **Explicável.** Cada recomendação traz as regras disparadas, a área e os termos em comum e a nota prevista — importante num contexto educacional, em que o aluno precisa confiar na sugestão.
- **Respeita requisitos acadêmicos.** O sistema não recomenda *ML Avançado* a quem não tem os pré-requisitos e indica a obra que os desbloqueia.
- **Escala.** Leitura em blocos, tipos compactos e álgebra esparsa processam as 10 milhões de interações num computador pessoal.
- **Avaliação honesta.** Divisão temporal, validação separada do teste, intervalos de confiança, um critério de significância para os pesos e um teto (oráculo) contra o qual medir.
- **Reprodutível e testado.** Um comando gera todos os resultados e este relatório; testes automatizados cobrem métricas, regras e modelos.

Pontos fracos

- **Os dados fornecidos não permitem avaliar a qualidade real.** Todos os ganhos medidos vêm do cenário simulado, cujas hipóteses favorecem os sinais que o sistema sabe usar.
- **Base de conhecimento manual.** Pré-requisitos, relevância curricular e o mapa termo → área foram escritos à mão; precisam de revisão por especialistas e de manutenção a cada mudança de currículo.

- **Filtragem colaborativa dependente de densidade.** Com cerca de 20 avaliações por material, a filtragem colaborativa não descobre sozinha que as 1.000 cópias são a mesma obra (no nível de obra fica atrás de conteúdo e conhecimento), é sensível à regularização (forte demais, os fatores colapsam) e não tem o que oferecer a quem não tem histórico.
- **Conteúdo raso.** Título e descrição têm poucas palavras; o TF-IDF erra com ambiguidade ("Redes" de computadores × "redes neurais") e não distingue as 1.000 cópias de uma obra.
- **Pesos globais.** Os pesos do híbrido são os mesmos para todos os alunos (exceto pelas regras de troca); alunos diferentes poderiam se beneficiar de combinações diferentes.
- **Métricas online simuladas.** CTR, engajamento e satisfação foram medidos com usuários simulados; a instrumentação existe, mas não houve usuários reais.

10 Sugestões de melhorias

Dados

- Corrigir a integridade referencial na origem e deduplicar o catálogo (uma obra com várias edições, não mil registros idênticos).
- Registrar sinais implícitos ricos — downloads, conclusão, retorno ao material, tempo de leitura real — e não só uma nota por interação.
- Enriquecer os metadados com ementa, palavras-chave e bibliografia das disciplinas.

Modelos

- Filtragem colaborativa para feedback implícito (ALS implícito ou BPR), que aprende com *quais* materiais o aluno escolhe e não só com a nota.
- *Embeddings* semânticos de texto (ex.: modelos de sentença em português) no lugar do TF-IDF, resolvendo sinônimos e ambiguidades.
- Pesos do híbrido aprendidos por *learning to rank* e personalizados por segmento de aluno.
- Base de conhecimento extraída dos planos de ensino (grafo de pré-requisitos das disciplinas) em vez de escrita à mão.
- Exploração com *bandits* contextuais para descobrir materiais novos e reduzir o viés de popularidade.

Avaliação e produção

- Teste A/B real: sortear o braço por aluno (não por sessão), definir CTR como métrica primária e satisfação e conclusão de materiais como secundárias, com métricas de proteção (tempo de carregamento, reclamações) e tamanho de amostra calculado antes do início.
- Métricas de novidade, serendipidade e equidade entre cursos.
- Servir o modelo por uma API (ex.: FastAPI) com pontuações pré-calculadas e monitoramento de desvio dos dados.

11 Demonstração funcional

```
pip install -r requirements.txt
python pipeline.py        # treina e avalia (≈ 15 min); gera os modelos em artefatos/
streamlit run app.py      # abre a demonstração no navegador
```

A demonstração tem quatro abas:

- **Recomendações** — escolha o cenário (dados fornecidos ou sinal controlado), o aluno (um exemplo de cada perfil, ou qualquer id até 100.000, inclusive sem cadastro), o modelo e o tamanho da lista. Mostra o perfil, as competências adquiridas pelo histórico, o histórico resumido e a lista com "Por que este material?" (regras, conteúdo e nota prevista). Os pesos do híbrido, o λ do MMR e a penalidade podem ser ajustados ao vivo. Inclui os alunos de comportamento mais parecido segundo a filtragem colaborativa e a pesquisa de satisfação.
- **Métricas offline** — as tabelas e figuras deste relatório.
- **Métricas online** — CTR, minutos por sessão e satisfação calculados dos eventos registrados na própria demonstração, e o teste A/B simulado.

- **Qualidade dos dados** — os números e figuras da Seção 2.

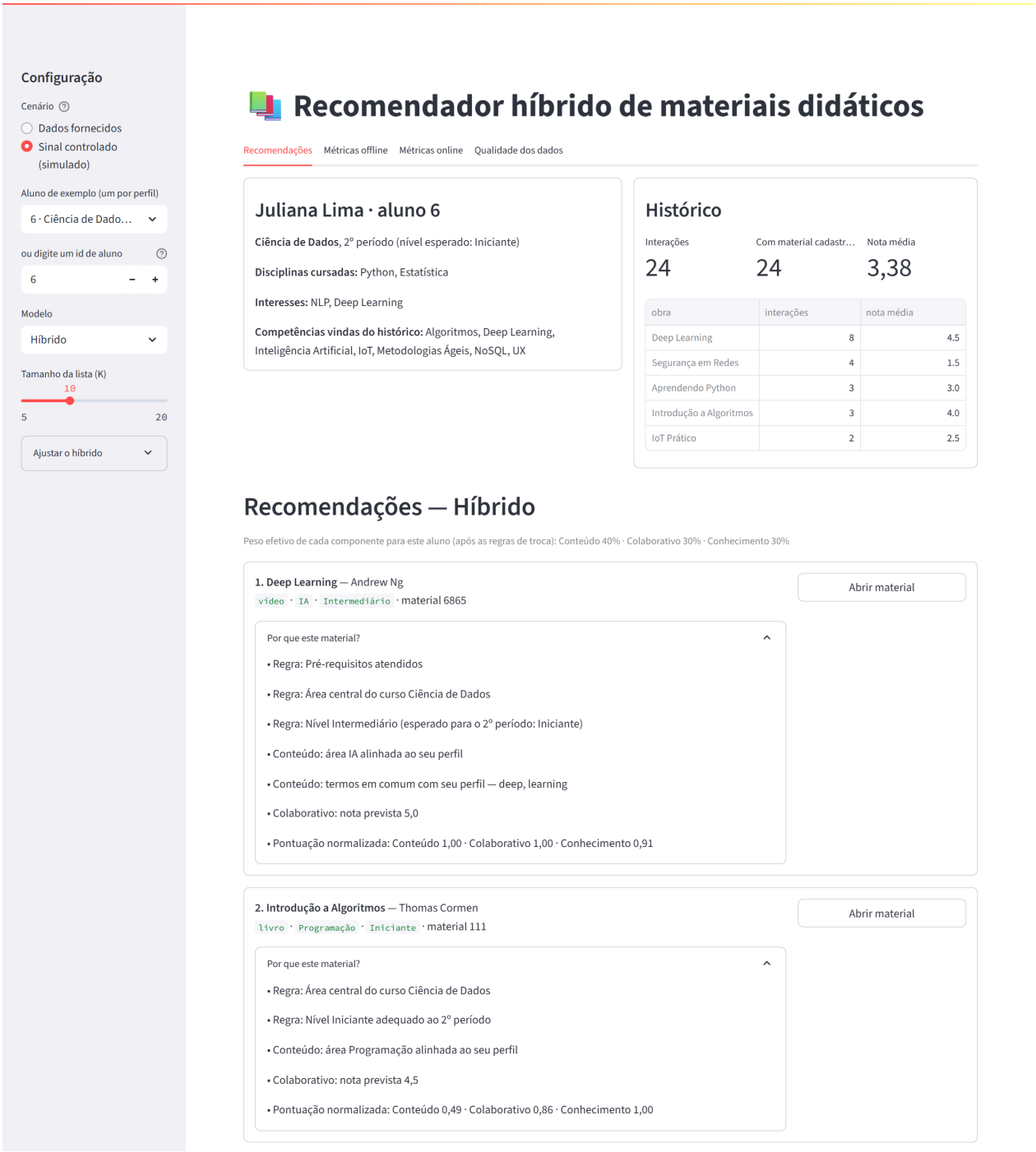


Figura 12: Aba de recomendações: o híbrido no cenário com sinal controlado, com as justificativas abertas

A configuração inicial também pode vir pela URL, o que permite compartilhar uma tela específica: `http://localhost:8501/?cenario=controlado&aluno=6&modelo=Híbrido`. Sem navegador, a mesma lógica está disponível na linha de comando:

```
python recomendar.py --aluno 2                          # top 10 do híbrido, com motivos
python recomendar.py --aluno 1 --regras --k 3           # regras para as 10 obras
python recomendar.py --aluno 6 --cenario controlado     # cenário com sinal
python recomendar.py --aluno 35000                      # aluno sem cadastro
```

12 Conclusão

O sistema implementa as três abordagens pedidas e as combina num híbrido que pondera, troca pesos conforme o que se sabe do aluno, respeita pré-requisitos e diversifica a lista, explicando cada recomendação. A análise dos dados mostrou que as interações fornecidas são ruído uniforme, com 98% delas sem cadastro correspondente; nesse cenário, o resultado correto é o que se obteve — nenhum modelo supera o acaso, e a validação impediu que ruído fosse confundido com padrão. No cenário com sinal controlado, cada componente capturou o sinal que deveria e o híbrido superou todos eles em ranqueamento, previsão de notas e no teste A/B simulado. O próximo passo de maior impacto não é um modelo melhor, e sim dados melhores: integridade referencial, catálogo sem duplicatas e sinais implícitos de uso.

Referências

- Burke, R. (2002). Hybrid recommender systems: survey and experiments. *User Modeling and User-Adapted Interaction*, 12(4), 331–370.
- Carbonell, J.; Goldstein, J. (1998). The use of MMR, diversity-based reranking for reordering documents and producing summaries. *SIGIR '98*.
- Cremonesi, P.; Koren, Y.; Turrin, R. (2010). Performance of recommender algorithms on top-N recommendation tasks. *RecSys '10*.
- Efron, B.; Tibshirani, R. (1993). *An Introduction to the Bootstrap*. Chapman & Hall.
- Felfernig, A.; Burke, R. (2008). Constraint-based recommender systems: technologies and research issues. *ICEC '08*.
- Herlocker, J. et al. (2004). Evaluating collaborative filtering recommender systems. *ACM TOIS*, 22(1), 5–53.
- Järvelin, K.; Kekäläinen, J. (2002). Cumulated gain-based evaluation of IR techniques. *ACM TOIS*, 20(4), 422–446.
- Kohavi, R.; Tang, D.; Xu, Y. (2020). *Trustworthy Online Controlled Experiments*. Cambridge University Press.
- Koren, Y.; Bell, R.; Volinsky, C. (2009). Matrix factorization techniques for recommender systems. *IEEE Computer*, 42(8), 30–37.
- Ricci, F.; Rokach, L.; Shapira, B. (orgs.) (2022). *Recommender Systems Handbook*. 3. ed. Springer.
- Rocchio, J. (1971). Relevance feedback in information retrieval. In: *The SMART Retrieval System*. Prentice-Hall.
- Zhou, Y. et al. (2008). Large-scale parallel collaborative filtering for the Netflix Prize. *AAIM 2008*.
- Ziegler, C.-N. et al. (2005). Improving recommendation lists through topic diversification. *WWW '05*.

Apêndice A — Precisão, recall e F1 para K = 5, 10 e 20

Tabela 17: Precisão, recall e F1 para K = 5, 10 e 20 nos dados fornecidos

Modelo	P@5	R@5	F1@5	P@10	R@10	F1@10	P@20	R@20	F1@20
Aleatório	0,0002	0,0006	0,0003	0,0002	0,0011	0,0004	0,0002	0,0025	0,0004
Popularidade	0,0001	0,0002	0,0001	0,0001	0,0007	0,0002	0,0002	0,0021	0,0003
Conteúdo	0,0002	0,0006	0,0003	0,0002	0,0009	0,0003	0,0002	0,0019	0,0003
Colaborativo	0,0001	0,0003	0,0002	0,0002	0,0009	0,0003	0,0001	0,0013	0,0002
Conhecimento	0,0002	0,0005	0,0003	0,0002	0,0011	0,0003	0,0002	0,0019	0,0003
Híbrido	**0,0001**	**0,0002**	**0,0002**	**0,0001**	**0,0006**	**0,0002**	**0,0001**	**0,0011**	**0,0002**

Tabela 18: Precisão, recall e F1 para K = 5, 10 e 20 no cenário controlado

Modelo	P@5	R@5	F1@5	P@10	R@10	F1@10	P@20	R@20	F1@20
Aleatório	0,0002	0,0003	0,0002	0,0002	0,0007	0,0003	0,0003	0,0017	0,0005
Popularidade	0,0007	0,0013	0,0009	0,0007	0,0028	0,0011	0,0007	0,0056	0,0012
Conteúdo	0,0008	0,0014	0,0010	0,0007	0,0029	0,0012	0,0007	0,0054	0,0012
Colaborativo	0,0012	0,0022	0,0016	0,0011	0,0043	0,0018	0,0011	0,0087	0,0020
Conhecimento	0,0006	0,0013	0,0008	0,0005	0,0023	0,0008	0,0005	0,0042	0,0009
Híbrido	**0,0014**	**0,0025**	**0,0018**	**0,0014**	**0,0054**	**0,0022**	**0,0013**	**0,0099**	**0,0023**
Oráculo (teto)	0,0041	0,0080	0,0054	0,0038	0,0150	0,0061	0,0033	0,0260	0,0059

Apêndice B — Reprodutibilidade

Todos os números deste relatório são gerados por `python pipeline.py` (sementes fixas em `recomendador/config.py`). As tabelas em `relatorio/tabelas/` e os valores citados no texto (`relatorio/variaveis.yml`) são reescritos a cada execução, e o documento é renderizado pelo Quarto em `.docx` e `.pdf`. Os resultados brutos ficam em `resultados/` (CSV e JSON) e os modelos treinados em `artefatos/`.